

Match the Git command to the GUI action

A live matching activity for the Git for Science workshop

How this works

In this workshop we drive Git through a graphical interface: buttons in the RStudio Git pane and on GitHub. Every one of those clicks runs a Git command underneath. Clicking the push button *is* git push.

Seeing that equivalence matters. Using the interface is not “cheating”, it is running the same commands a terminal user types. It is also the bridge to the agentic-coding follow-up, where an agent drives Git by naming these same commands.

The activity (10 minutes, in pairs): the left column lists Git commands, numbered. The right column describes GUI actions, lettered and out of order. Draw a line from each command to its matching action, or write the letter next to the number. Then we compare answers as a group.

Learner sheet

Git command	GUI action (out of order)
1. <code>git clone <url></code>	A. Click the down-arrow (pull) button in the RStudio Git pane
2. <code>git add <file> / stage</code>	B. Write a commit message, then click Commit
3. <code>git commit -m "..."</code>	C. On GitHub, click Add file > Create new file or Upload files , then Commit changes
4. <code>git push</code>	D. Copy the HTTPS URL, then File > New Project > Version Control > Git in RStudio
5. <code>git pull</code>	E. Click the up-arrow (push) button in the RStudio Git pane
6. <code>git status</code>	F. Tick the checkbox next to a file in the RStudio Git pane
7. <code>git log</code>	G. On GitHub, open the Pull requests tab and click New pull request
8. <code>git branch <name> / checkout</code>	H. Read the file list and status flags shown in the RStudio Git pane
9. <code>git add + git commit</code> (on GitHub)	I. Click the branch dropdown in the Git pane and choose New Branch
10. open a pull request	J. Click History (the clock icon) in the RStudio Git pane

Pairs: 1

_____ 2_____ 3_____ 4_____ 5_____ 6_____ 7_____ 8_____ 9_____ 10_____

Facilitator answer key

Do not print this page for learners.

Command	Answer	GUI action
1. <code>git clone <url></code>	D	File > New Project > Version Control > Git
2. <code>git add / stage</code>	F	Tick the checkbox next to a file in the Git pane
3. <code>git commit -m "..."</code>	B	Write a message, click Commit
4. <code>git push</code>	E	Up-arrow (push) button in the Git pane
5. <code>git pull</code>	A	Down-arrow (pull) button in the Git pane
6. <code>git status</code>	H	Read the file list and status flags in the Git pane
7. <code>git log</code>	J	History (clock icon) in the Git pane
8. <code>git branch / checkout</code>	I	Branch dropdown > New Branch
9. <code>git add + commit</code> on GitHub	C	Add file / Upload files, then Commit changes
10. open a pull request	G	Pull requests tab > New pull request

Talking points while reviewing

- Every button press ran a real command. Nothing was hidden or “easier”; it was the same operation.
- `git add` and `git commit` are two steps. In the RStudio pane you see this clearly: tick the box (add), then Commit. On GitHub the web editor bundles both into one “Commit changes” click.
- A pull request is not a Git command in the same sense; it is a GitHub feature built on top of branches you pushed. Good moment to name where Git ends and GitHub begins.
- Tie-in to the agentic follow-up: an agent driving Git speaks these same command names. Knowing the mapping is what lets you read and check what the agent does.